



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

**ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №8
«КЛАСТЕРИЗАЦИЯ ДАННЫХ»**

по дисциплине «Языки программирования для статистической обработки
данных»

Студент группы

ИНБО-20-23. Боргачев Т.М.

(подпись)

Преподаватель

Трушин СМ

(подпись)

Москва 2025 г.

ВВЕДЕНИЕ

Цель практики: научиться проводить кластеризацию данных с использованием методов K-means и иерархической кластеризации в Python, R, а также визуализировать результаты кластерного анализа.

Задачи практики:

1. Выполнить кластеризацию методом K-means:
 - a. Python: использование библиотеки sklearn.
 - b. R: функция kmeans.
2. Провести иерархическую кластеризацию:
 - c. Python: библиотека `scipy.cluster.hierarchy`.
 - d. R: функции `hclust`, `dendrogram`.
3. Проанализировать результаты кластеризации:
 - e. Интерпретация кластеров (центроиды, количество объектов в каждом кластере).
2. Сравнение кластеров, полученных разными методами.
4. Визуализировать результаты кластеризации:
 - b. Python: графики кластеров с использованием `matplotlib` и `seaborn`.
 - c. R: графическое представление дендрограмм и кластеров.
5. Сравнить удобство выполнения кластерного анализа в Python, R.

РЕЗУЛЬТАТЫ РАБОТЫ

Подготовка данных

Из всех данных отберем только числовые признаки, которые будут атрибутами для кластеризации.

В датасете имеются следующие поля:

1. Id
2. Name
3. Type 1
4. Type 2
5. Total
6. HP
7. Attack
8. Defense
9. Sp. Atk
10. Sp. Def
11. Speed
12. Generation
13. Legendary

Для кластеризации будем использовать признаки с 6го по 11ый. Применим к каждому из них стандартизацию по средством StandardScaler и scale.

Код для предварительной подготовки данных на Python представлен в листинге 1.

Листинг 1 – Предобработка данных в Python

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("Pokemon.csv")
df.head()
df_features = df[["HP", "Attack", "Defense", "Sp. Atk", "Sp. Def", "Speed"]]
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df_features)
df["Type 1"].unique().size
```

Код для предварительной подготовки данных на R представлен в листинге 2.

Листинг 2 - Предобработка данных в R

```
library(ggplot2)
df <- read.csv("Pokemon.csv", header=TRUE)
# Select the desired columns from the data frame
selected_columns <- c("HP", "Attack", "Defense", "Sp..Atk", "Sp..Def", "Speed")
existing_columns <- intersect(selected_columns, colnames(df))
df_features <- df[, existing_columns]
print(df_features)
print(head(df))
str(df)
summary(df)
# Масштабирование
df_scaled <- scale(df_features)
set.seed(42) # фиксация рандома
```

Кластеризация методом K-means

Цель: разделить данные на k кластеров так, чтобы сумма квадратов расстояний от точек до центроидов (центров кластеров) была минимальна.

Начальное приближение: изначально центроиды выбираются случайно (или посредством определённых алгоритмов инициализации).

Алгоритм:

1. Рассчитать расстояния от каждой точки до всех центроидов.
2. Присвоить точку ближайшему центроиду.
3. Пересчитать координаты центроидов как среднее по всем точкам
4. кластера.
5. Повторять, пока не достигнута сходимость или не выполнено заданное
6. число итераций.

Код для K-means кластеризации на Python представлен в листинге 3.

Листинг 3 – K-means в Python

```
k = 3 # - Если мы учитываем только характеристики, то можно либо характеризовать
это как 3 уровня эволюции
#либо просто некий топ
kmeans = KMeans(n_clusters=k, random_state=42)
kmeans.fit(df_scaled)
# Предсказанные кластеры
labels = kmeans.labels_
# Центроиды
centroids = kmeans.cluster_centers_
print("Кластеры для объектов:\n", labels)
print("Координаты центроидов:\n", centroids)
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
df_pca = pca.fit_transform(df_scaled)
plt.scatter(df_pca[:,0], df_pca[:,1], c=labels)
plt.title("K-Means (2 компоненты PCA)")
plt.show()
```

Визуализация 3х кластеров на Python представлена на рис. 1.

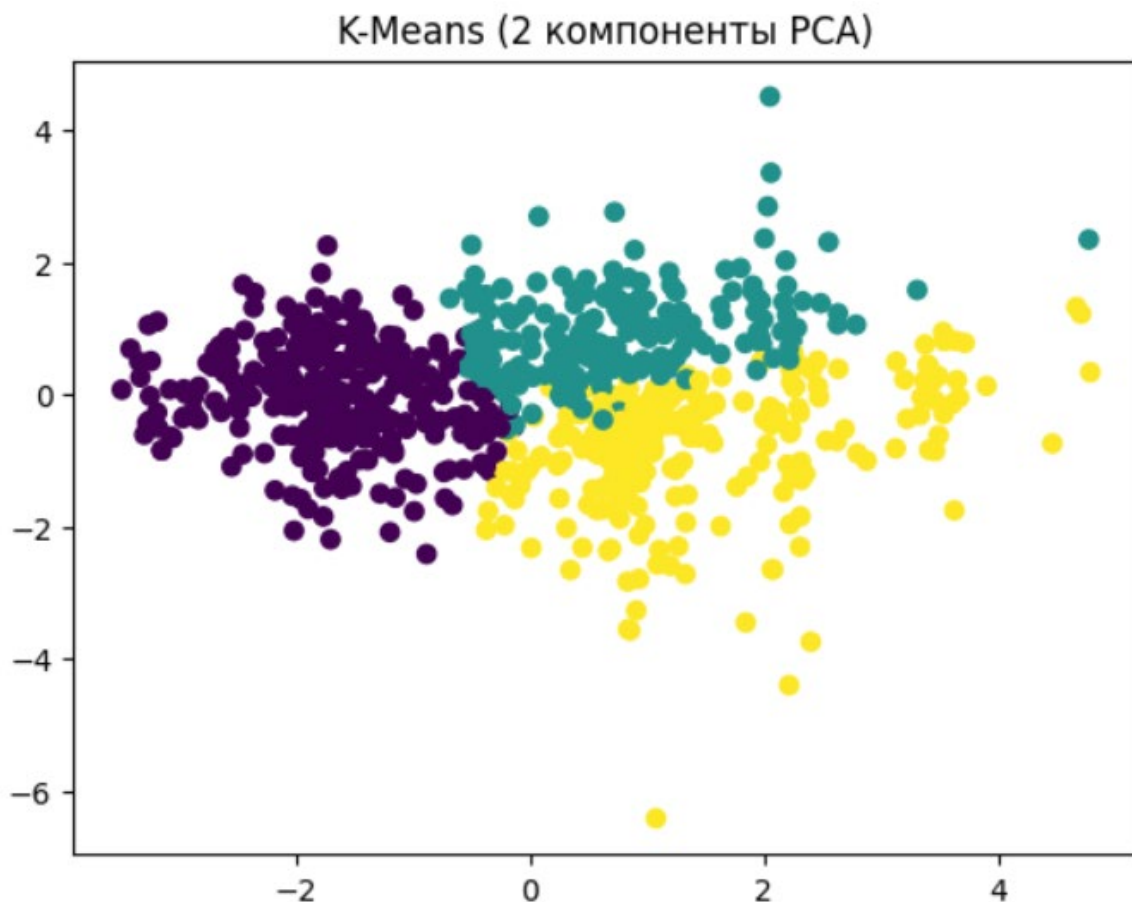


Рисунок 1 – Визуализация трех кластеров в Python

Код для K-means кластеризации на R представлен в листинге 4.

Листинг 4 – K-means в R

```
k <- 3 # я буду думать что это эволюции
kmeans_result <- kmeans(df_scaled, centers = k, nstart = 10)
# nstart - количество случайных инициализаций
kmeans_result$cluster # вектор принадлежности кластеру
kmeans_result$centers # координаты центроидов
kmeans_result$tot.withinss # суммарная внутрикластерная сумма квадратов

library(ggplot2)
pca_res <- prcomp(df_scaled)
df_pca <- data.frame(pca_res$x[,1:2], Cluster =
factor(kmeans_result$cluster))
ggplot(df_pca, aes(x=PC1, y=PC2, color=Cluster)) +
geom_point() +
ggtitle("K-means Clustering (PCA projection)")
```

Визуализация 3х кластеров на R представлена на рис. 2.

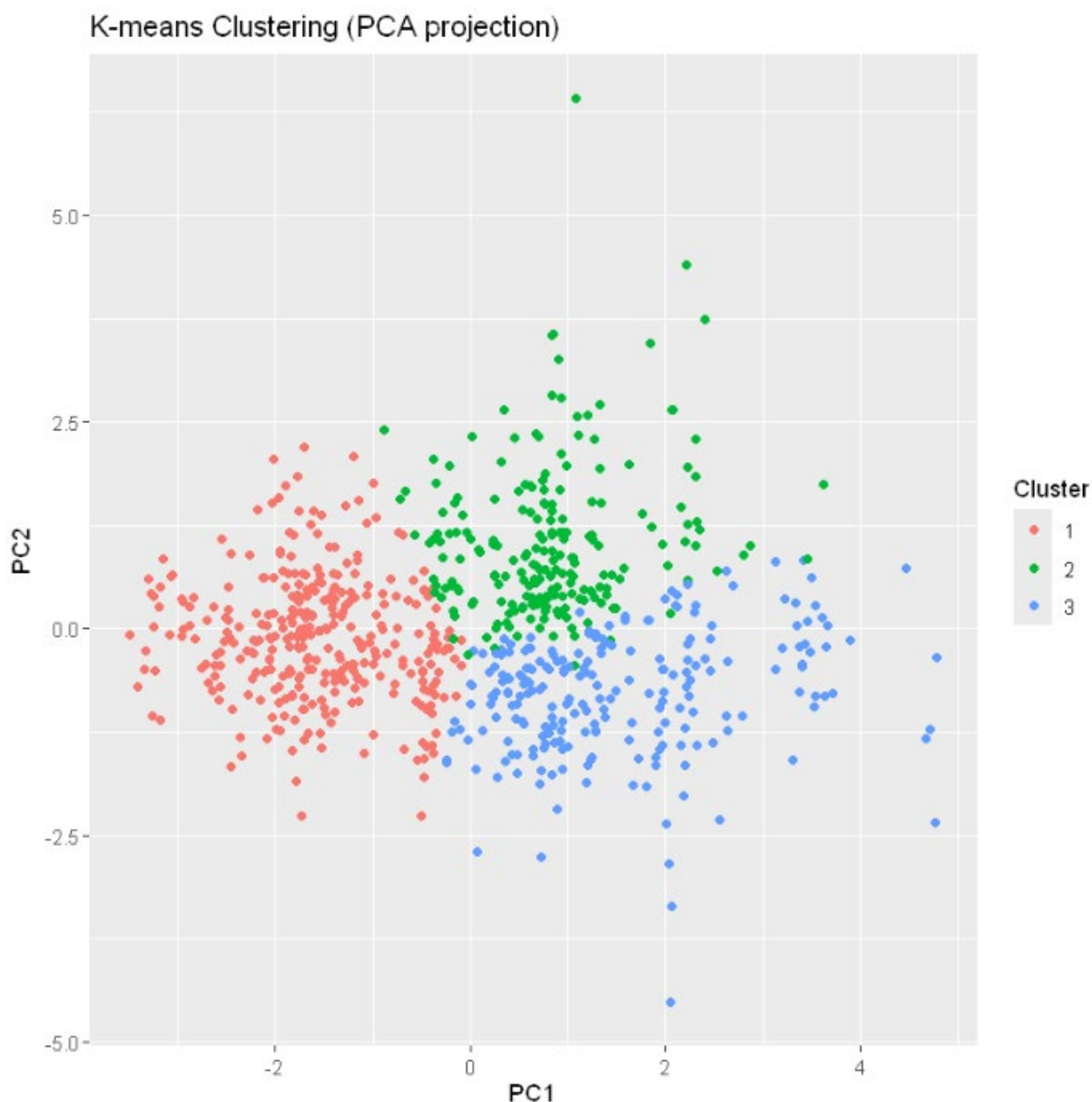


Рисунок 2 - Визуализация трех кластеров в R

Таким образом, все записи были поделены на 3 группы, в зависимости от различных характеристик, что хорошо наблюдаемо на графике рассеивания. Можно предположить, что это группы с наиболее выраженными отличительными характеристиками – например высокая скорость и низкая защита, высокая защита и низкая атака или что-то подобное. В то же время, есть вероятность что распределение было по принципу – самые слабые, средние, самые сильные. То есть разделение на 3 группы силы.

Иерархическая кластеризация

Код для иерархической кластеризации на Python представлен в листинге 5.

Листинг 5 – Иерархическая кластеризация в Python

```
import scipy.cluster.hierarchy as sch
# linkage: метод объединения, например "ward" (минимизация роста внутрикластерной
суммы квадратов)
Z = sch.linkage(df_scaled, method='ward')
# Построение дендрограммы
plt.figure(figsize=(10, 7))
dn = sch.dendrogram(Z)
plt.title("Иерархическая кластеризация (дендрограмма)")
plt.show()
# Можно выделить 18 классов - типы покемонов. Не факт что они совпадут, но такую
задачу поставить можно
clusters = fcluster(Z, t=18, criterion='maxclust')
print(clusters)
```

Визуализация дендрограммы на Python представлена на рис. 3.

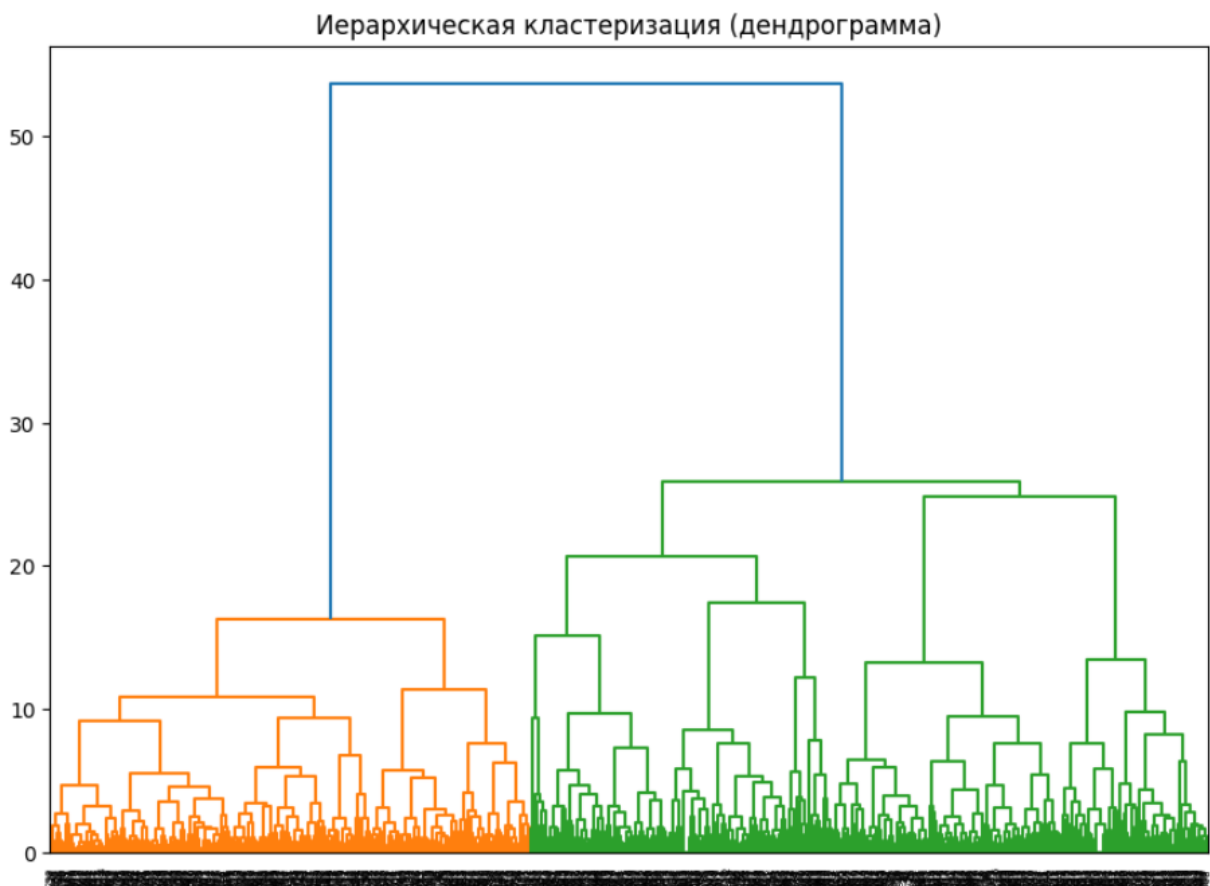


Рисунок 3 – Дендрограмма в Python

Код для иерархической кластеризации на R представлен в листинге 6.

Листинг 6 – Иерархическая кластеризация в R

```
dist_matrix <- dist(df_scaled, method="euclidean") # матрица расстояний
hc <- hclust(dist_matrix, method="ward.D2") # ward.D2
# Построение дендрограммы
plot(hc, main="Иерархическая кластеризация", xlab="", sub="")
# 18 Типов (будем считать что он 1 у всех)
clusters <- cutree(hc, k=18)
table(clusters)
```

Визуализация дендрогаммы на R представлена на рис. 4.

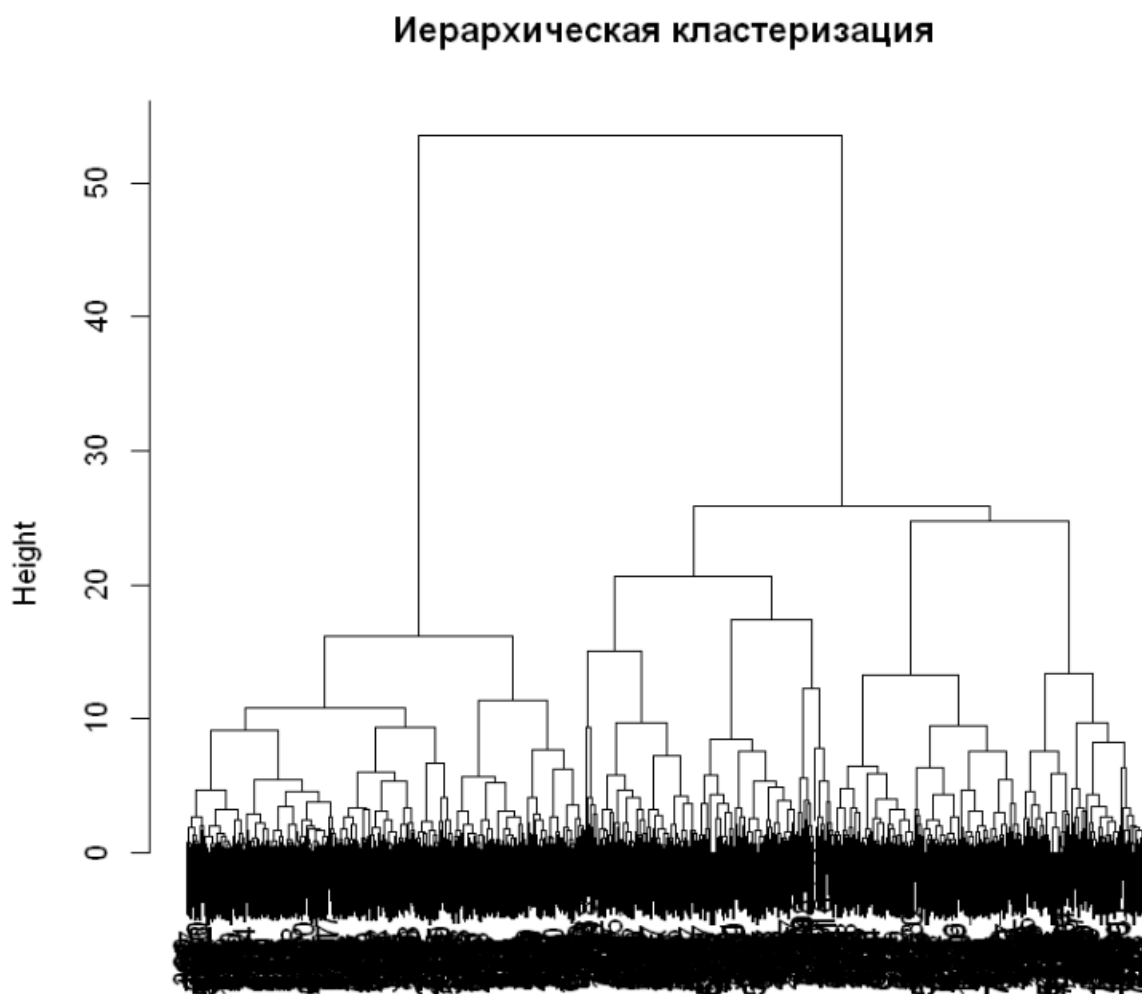


Рисунок 4 – Дендрогамма в R

По иерархической диаграмме кластеров можно четко разграничить 3 кластера, 6 кластеров и последняя прямая – около 20 кластеров. Далее начинается слишком большое и близкое по расстоянию дробление.

Анализ результатов кластеризации

На основе визуализации трех кластеров с использованием PCA (см. рис. 2), можно сделать следующие наблюдения:

- Кластер 1 (красный):
 - Точки этого кластера расположены преимущественно в левой части графика.
 - Характеризуется низкими значениями первой главной компоненты (PC1) и второй главной компоненты (PC2).
 - Предположительно, этот кластер содержит объекты с более низкими значениями признаков (например, низкая скорость и защита).
- Кластер 2 (зеленый):

- Точки этого кластера находятся в центральной части графика.
- Характеризуется средними значениями PC1 и PC2.
- Этот кластер, вероятно, содержит объекты со средними характеристиками (баланс между скоростью, атакой, защитой и другими параметрами).
- Кластер 3 (синий):
 - Точки этого кластера расположены преимущественно в правой части графика.
 - Характеризуется высокими значениями PC1 и относительно низкими значениями PC2.
 - Предположительно, этот кластер содержит объекты с высокими значениями некоторых признаков (например, высокая скорость или атака).

Визуализация кластеров и сравнение инструментов

Код для конечной визуализации кластеров на Python представлен в листинге 7.

Листинг 7 – 3D-визуализация в Python

```
# Снижение размерности до 3D с помощью PCA
pca = PCA(n_components=3)
data_3d = pca.fit_transform(df_scaled)

# Создание 3D-графика
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    data_3d[:, 0], # Первая главная компонента
    data_3d[:, 1], # Вторая главная компонента
    data_3d[:, 2], # Третья главная компонента
    c=clusters,   # Цвета точек соответствуют кластерам
    cmap='viridis', # Цветовая карта
    s=50,         # Размер точек
    alpha=0.8     # Прозрачность
)

# Добавление подписей осей
ax.set_xlabel('Principal Component 1')
ax.set_ylabel('Principal Component 2')
ax.set_zlabel('Principal Component 3')

# Добавление легенды
legend = ax.legend(*scatter.legend_elements(), title="Clusters")
ax.add_artist(legend)

# Показать график
plt.show()
```

Визуализация 18 кластеров на Python представлена на рис. 5.

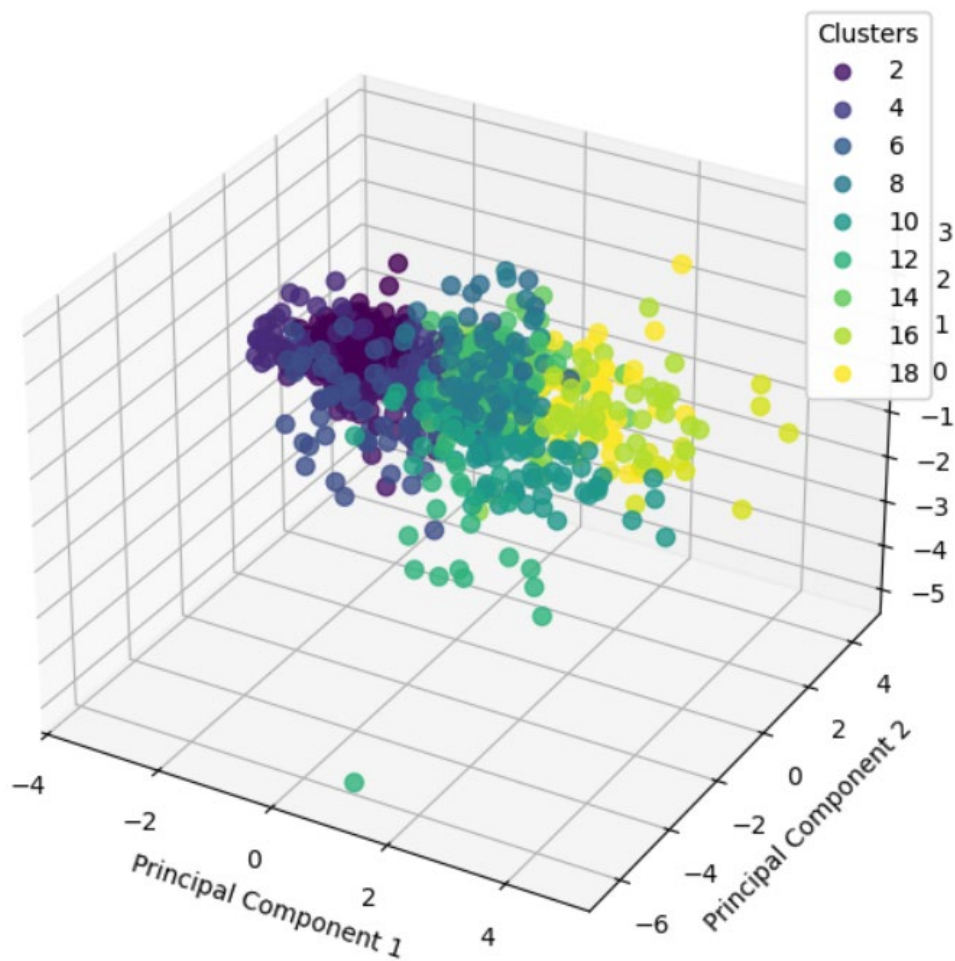


Рисунок 5 – Визуализация 18 кластеров в Python

Код для конечной визуализации кластеров на R представлен в листинге 8.

Листинг 8 – 3D-визуализация в R

```
library(plotly)
library(scatterplot3d)

pca_result <- prcomp(df_scaled, scale. = TRUE)
data_3d <- pca_result$x[, 1:3] # Берем первые три главные компоненты

# Создание 3D-графика с помощью plotly
plot_ly(data = data.frame(data_3d, Cluster = as.factor(clusters))) %>%
  add_trace(
    x = ~PC1, y = ~PC2, z = ~PC3,
    color = ~Cluster,
    colors = viridis::viridis(length(unique(clusters))), # Цветовая карта
    type = "scatter3d",
    mode = "markers",
    marker = list(size = 5, opacity = 0.8)
  ) %>%
  layout(
    scene = list(
      xaxis = list(title = "Principal Component 1"),
      yaxis = list(title = "Principal Component 2"),
      zaxis = list(title = "Principal Component 3")
    ),
    title = "3D PCA Plot with Clusters"
  )
```

Визуализация 18 кластеров на R представлена на рис. 6.

3D PCA Plot with Clusters

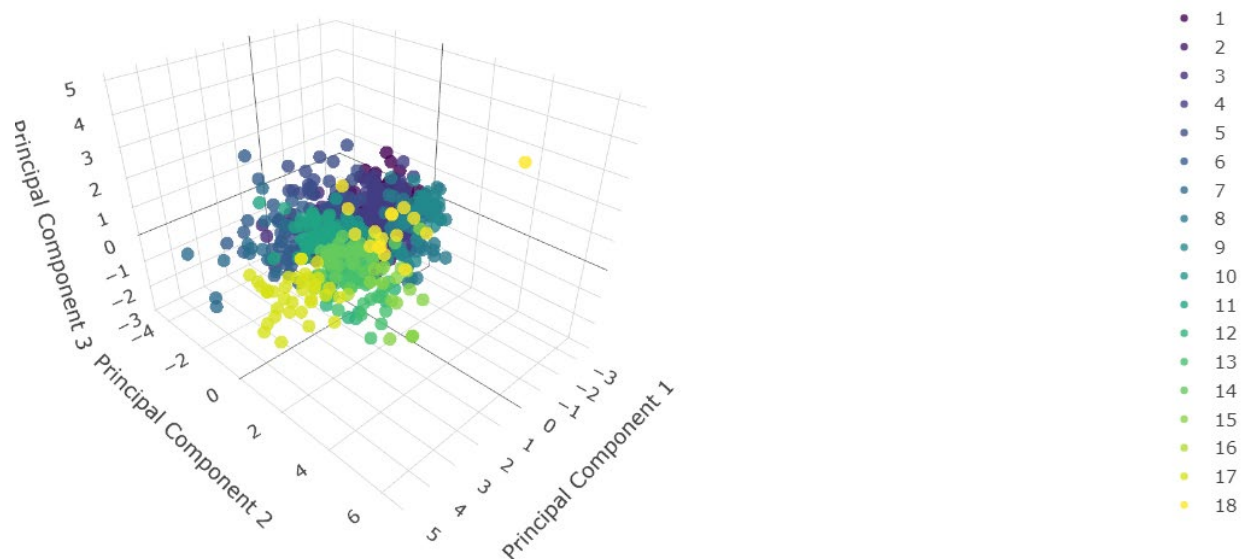


Рисунок 6 – Визуализация 18 кластеров в R

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы были проведены следующие шаги:

1. Подготовка данных: отбор числовых признаков и их стандартизация.
2. Кластеризация методом K-means в Python и R.
3. Проведение иерархической кластеризации в Python и R.
4. Визуализация результатов кластеризации с использованием PCA и дендрограмм.
5. Анализ и сравнение результатов кластеризации.

Выводы:

- Метод K-means позволил эффективно разделить данные на три кластера, каждый из которых характеризуется своими уникальными свойствами.
- Иерархическая кластеризация предоставила дополнительную информацию о структуре данных на разных уровнях детализации.
- Визуализация с использованием PCA помогла наглядно представить результаты кластеризации.

Удобство использования Python и R:

- Python предлагает широкие возможности для кластеризации и визуализации благодаря мощным библиотекам.
- R обладает встроенной поддержкой кластеризации и простыми средствами визуализации.

Рекомендации:

- Для задач с известным числом кластеров рекомендуется использовать K-means.
- Для исследовательских задач рекомендуется использовать иерархическую кластеризацию.
- Для дальнейших исследований целесообразно использовать метрики качества кластеризации для определения оптимального числа кластеров.